

PROJECT REPORT

Parallel Cellular Automaton Model for Wildfire Propagation using MPI

Parallel and Distributed Computing using MPI

K Sri Praneetha — 245805006

K Ruthika Reddy — 245805344

Implementation: C with MPI (OpenMPI / MPICH)

Sequential and parallel versions, correctness verification and performance study

Abstract

Wildfire spread is a classic neighbour-driven simulation: the future state of every cell of a landscape depends on the current state of the cells around it, on the wind and on the vegetation. This project models wildfire propagation with a two-dimensional cellular automaton in which every cell is unburned, burning or burnt, and parallelises the simulation with MPI. The grid is divided row-wise into strips, one per process (domain decomposition); because fire spreading across a strip boundary depends on a row owned by another process, every process exchanges its edge rows with its neighbours at every timestep using MPI_Sendrecv (halo exchange). A sequential version of the same model was also implemented. Both versions share one update function and use a position-based deterministic random number generator, so the parallel result is bit-identical to the sequential result for any number of processes; this is verified automatically. Execution time, speedup, efficiency and the share of time spent communicating were measured with MPI_Wtime.

1. Introduction

A wildfire is a large, uncontrolled fire that spreads across a natural landscape such as a forest or grassland. How quickly it spreads and in which direction depends mainly on the wind and on the type and density of vegetation it passes through. Predicting the spread is a real problem for forest services and disaster-management agencies, who use such predictions for evacuation planning and for allocating firefighting resources.

This project models the spread with a cellular automaton: the landscape is a two-dimensional grid whose cells have a state (unburned, burning or burnt). In every timestep each cell is updated using only the states of its immediate neighbours, the local wind direction and the terrain (fuel) value of the cell. Repeating this simple local rule over the whole grid for many timesteps produces a realistic, wind-shaped fire front.

MPI (Message Passing Interface) is used to divide the grid among several processes and to exchange boundary information between neighbouring processes at every timestep, so the simulation is computed correctly and faster than a single sequential process could manage.

2. Problem Statement

A sequential simulation processes every cell of the grid, one at a time, for every timestep. As the grid size and the number of timesteps grow, the total work grows very quickly (a 2000×2000 grid for 5000 timesteps is 20 billion cell updates) and a single process cannot use the multiple cores of a modern machine. The grid can be divided into regions that are updated concurrently by separate MPI processes, but fire at the edge of one process's region depends on the current state of the neighbouring region owned by a different process. The processes must therefore exchange boundary information at every timestep for the result to remain correct. This project implements that repeated communication pattern with MPI and compares sequential and parallel execution time to measure the speedup actually achieved.

3. Objectives

- **Use MPI for parallel wildfire simulation** — set up an MPI program in which multiple processes run together and share grid data.
- **Divide the grid among processes (domain decomposition)** — split the 2D grid row-wise into (nearly) equal strips, one per process.

- **Exchange boundary information** — before every timestep each process exchanges its edge rows with its neighbours (halo exchange) so spread across a process boundary is computed correctly.
- **Model realistic fire spread** — update each cell from its neighbours, a configurable wind direction and strength, and a per-cell vegetation (fuel) value.
- **Compare sequential and parallel execution time** — implement a sequential and an MPI version and measure how much faster the parallel version is for the same grid size and number of timesteps.
- **Verify correctness** — show that the parallel result is exactly the same as the sequential result.

4. Methodology

4.1 Overall workflow

The program follows the pipeline shown in Figure 1. The initial grid is created once by the root process (rank 0), which also acts as coordinator: it distributes the grid, and at the end reassembles and saves the result. All other work — the repeated halo exchange and cell update — is done by all processes simultaneously.

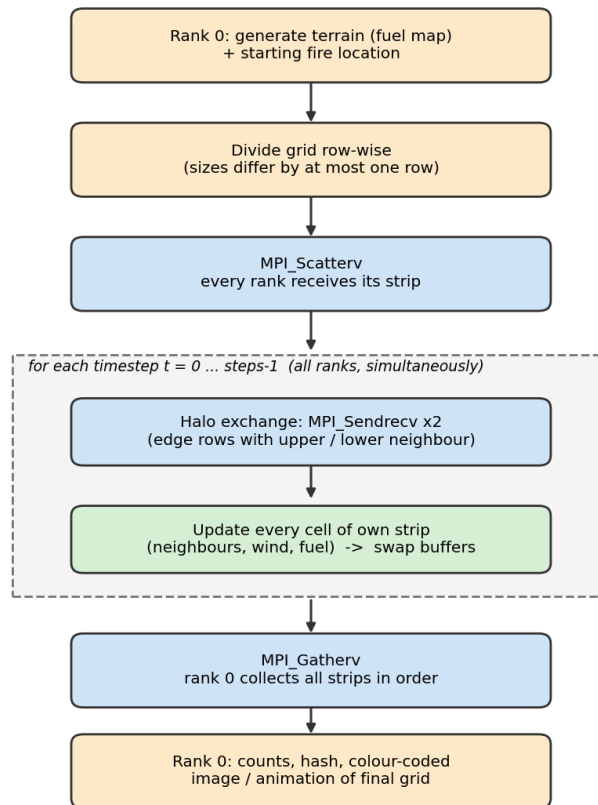


Figure 1: Workflow of the parallel wildfire simulation.

4.2 Domain decomposition

The H rows of the grid are divided among p processes. Every process gets $\lfloor H/p \rfloor$ rows and the first $(H \bmod p)$ processes get one extra row, so strip sizes differ by at most one row and any grid height works, not only multiples of p . Because of this, MPI_Scatterv and MPI_Gatherv (variable counts and displacements) are used instead of plain MPI_Scatter and MPI_Gather. For example a grid with 2000

rows on 4 processes gives four strips of 500 rows; a grid with 203 rows on 4 processes gives strips of 51, 51, 51 and 50 rows.

Row-wise decomposition with halo rows (4 processes)

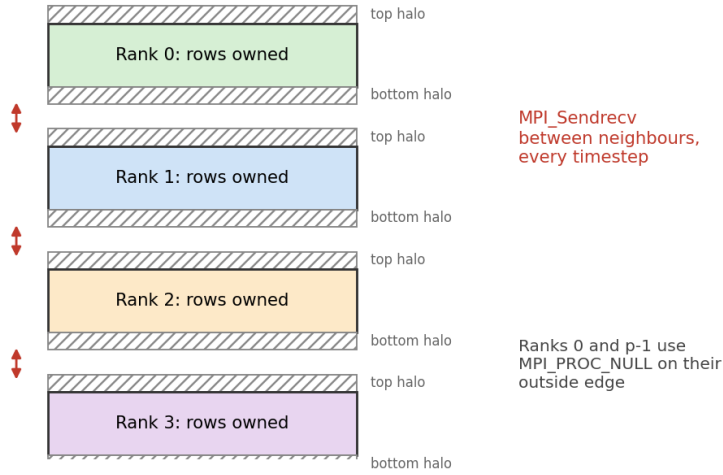


Figure 2: Row-wise decomposition. Each process stores its own rows plus one halo row above and below.

4.3 Halo exchange

A process needs the state of the row directly above its first row and directly below its last row to update its edge cells. Each process therefore stores rows+2 rows: one top halo row, its own rows, and one bottom halo row. At the start of every timestep two MPI_Sendrecv calls fill the halo rows: the first sends the process's first row upward while receiving the bottom halo from the process below; the second sends the last row downward while receiving the top halo from the process above. The first and last processes use MPI_PROC_NULL as the missing neighbour, which turns the call into a no-op and leaves that halo as zeros (= 'unburned'), so no special cases are needed. Using MPI_Sendrecv, which combines the send and the receive in one call, avoids the deadlock that can occur when every process first calls a blocking send. The terrain (fuel) map never changes, so it is distributed once and is not exchanged.

```
/* HALO EXCHANGE: send first real row up / receive bottom halo from below */
MPI_Sendrecv(cur + (size_t)1 * W, W, MPI_UNSIGNED_CHAR, up, 0,
             cur + (size_t)(rows + 1) * W, W, MPI_UNSIGNED_CHAR, down, 0,
             comm, MPI_STATUS_IGNORE);
/* send last real row down / receive top halo from above */
MPI_Sendrecv(cur + (size_t)rows * W, W, MPI_UNSIGNED_CHAR, down, 1,
             cur, W, MPI_UNSIGNED_CHAR, up, 1,
             comm, MPI_STATUS_IGNORE);
```

5. Fire Model

Cell states are 0 = unburned, 1 = burning and 2 = burnt. A separate read-only fuel map assigns every cell a vegetation value from 0 (water or bare rock, cannot burn) to 255 (dense vegetation). The map is generated from smooth value noise and contains lakes and a river that act as firebreaks. All cells are updated from the previous timestep's grid into a second grid (double buffering), and the two grids are swapped after each step.

Table 1: Cell update rules

Current state	Rule for the next timestep
Unburned, fuel = 0	Stays unburned (water / rock).
Unburned, fuel > 0	For each of the 8 neighbours that is burning, an ignition probability $p = \text{base} \times \text{fuel} \times \text{wind-factor}$ is computed. The probabilities combine as $P(\text{ignite}) = 1 - \prod(1 - p_i)$ and one random number decides whether the cell starts burning.
Burning	Burns out (becomes burnt) with probability $0.06 + 0.30 \cdot (1 - \text{fuel})$; denser fuel therefore burns longer.
Burnt	Stays burnt.

The wind factor for a neighbour is $\exp(k \cdot \cos \theta)$, where θ is the angle between the wind direction and the direction in which the fire would travel from that neighbour into the cell. It is larger than 1 downwind, smaller than 1 upwind, and equal to 1 everywhere when the wind strength k is 0. The wind direction is given in degrees (0 = blowing toward north, 90 = east, 180 = south, 270 = west).

5.1 Deterministic random numbers

Fire ignition and burn-out are random events. If an ordinary random-number stream (`rand()`) were used, the numbers drawn for a cell would depend on how many cells were processed before it, i.e. on how the grid is divided among processes, and the parallel result could never be compared with the sequential one. Instead, the random number for a cell is a hash function of (global row, column, timestep, purpose). It depends only on the cell and the timestep, never on the process that computes it. Together with the halo exchange this makes the parallel simulation produce exactly the same grid as the sequential one.

6. MPI Concepts Used

Table 2: MPI functions used

MPI function	Purpose in this project
MPI_Init / MPI_Finalize	Start up and cleanly shut down the MPI environment.
MPI_Comm_rank	Gives each process its ID, which determines which strip of the grid it owns.
MPI_Comm_size	Number of processes, needed to divide the grid.
MPI_Bcast	Sends the (parsed) parameters and lookup tables from rank 0 to all ranks.
MPI_Scatterv	Sends each process its strip of the initial grid and of the fuel map (strips may differ by one row).
MPI_Sendrecv	The halo exchange: edge rows are swapped with the upper and lower neighbour at every timestep.
MPI_PROC_NULL	Replaces the non-existent neighbour of the first and last process.
MPI_Barrier	Makes all processes start the timed loop together so timing is fair.
MPI_Wtime	Wall-clock time; used to measure communication and computation time separately.
MPI_Reduce (MPI_MAX)	The parallel time is the time of the slowest process.
MPI_Gatherv	Collects the final strips on rank 0 in rank order.

7. Implementation

Table 3: Project files

File	Description
src/fire_common.h	Terrain generator, deterministic random numbers, the cell-update function <code>fire_step()</code> , colour image (PPM) writer. Included by both programs, so both simulate the same model.
src/fire_seq.c	Sequential baseline: one process updates the whole grid.
src/fire_mpi.c	MPI version: decomposition, Scatterv, halo exchange, update, Gatherv, timing.
scripts/run_tests.sh	Correctness tests (parallel output must equal sequential output).
scripts/run_bench.sh	Timing experiments for several grid sizes and process counts.
scripts/visualize.py	Snapshot montage / animated GIF, progress plot, speedup and efficiency plots.
scripts/make_report.py	Generates this report from the measured result files.

Both programs are compiled with `gcc / mpicc` using `-O2 -ffp-contract=off` so floating-point evaluation is identical in the two programs. The programs accept the grid size, number of timesteps, random seed, wind direction and strength, base ignition probability and snapshot interval on the command line.

Timed region. Only the halo exchange and the cell updates of all timesteps are timed. Terrain generation, the scatter, writing snapshots and final output are excluded. Each process accumulates its communication and computation time with `MPI_Wtime`; the reported parallel time is the maximum over all processes (`MPI_Reduce` with `MPI_MAX`), since the slowest process determines when the simulation is finished. The sequential program times exactly the same region with `clock_gettime` (`CLOCK_MONOTONIC`); it does not need MPI.

8. Correctness Verification

Correctness is checked in three ways.

1. **Hash comparison** — every run prints a 64-bit FNV-1a hash of the final grid; the hash printed by the MPI program must equal the hash printed by the sequential program.
2. **Byte-level comparison** — the full final grid written by the sequential and the MPI program is compared with the `cmp` command.
3. **Awkward configurations** — tests use several process counts (1, 2, 3, 4, 5, 7, 8), several wind directions and strengths, and grid heights such as 103, 131, 157, 211 that are not divisible by most process counts.

Result of 'make test': ALL 28 CHECKS PASSED: parallel output == sequential output

In addition, the hashes of all runs of the performance experiment of Section 9 were compared: for every grid size, the sequential run and all MPI runs (all process counts and all repetitions) produced the identical hash.

9. Experimental Results

9.1 Setup

Table 4: Experimental environment

Item	Value
Processor	Intel(R) Xeon(R) Processor @ 2.10GHz
Cores available (nproc)	1
MPI implementation	mpirun (Open MPI) 4.1.6
Operating system	Linux 6.18.44-fc-v77
Date of measurement	2026-10-07
Timesteps per run	300
Grid sizes	500 × 500, 1000 × 1000
Repetitions	3 per configuration; the best (smallest) time is reported

IMPORTANT — measurement limitation. This machine reports only 1 usable core(s), but some runs used up to 4 processes. Rows marked * are oversubscribed: several MPI processes shared one core and were time-sliced by the operating system, so their time (and the large 'communication %', which then mostly measures waiting for a descheduled neighbour) does NOT represent real parallel speedup. These runs demonstrate that the program works for every process count and gives identical results; genuine speedup must be measured on a machine with at least as many cores as processes. Re-run 'make bench' there and then 'make report' — this section is rebuilt automatically from the new measurements.

9.2 Execution time, speedup and efficiency

Speedup is $S = T_{\text{seq}} / T_{\text{par}}$ and efficiency is $E = S / p$, where T_{seq} is the time of the sequential program and T_{par} the time of the MPI program with p processes (maximum over processes). 'Comm %' is the share of the run spent in the halo exchange.

Table 5: Measured times (300 timesteps); * = more processes than cores

Grid	Processes	T_{seq} (s)	T_{par} (s)	Speedup	Efficiency	Comm %
500 × 500	1	0.647	0.594	1.09	1.09	0.0
500 × 500	2*	0.647	0.538	1.20	0.60	53.2
500 × 500	4*	0.647	0.578	1.12	0.28	82.6
1000 × 1000	1	2.528	2.672	0.95	0.95	0.0
1000 × 1000	2*	2.528	2.448	1.03	0.52	38.0
1000 × 1000	4*	2.528	2.478	1.02	0.26	73.5

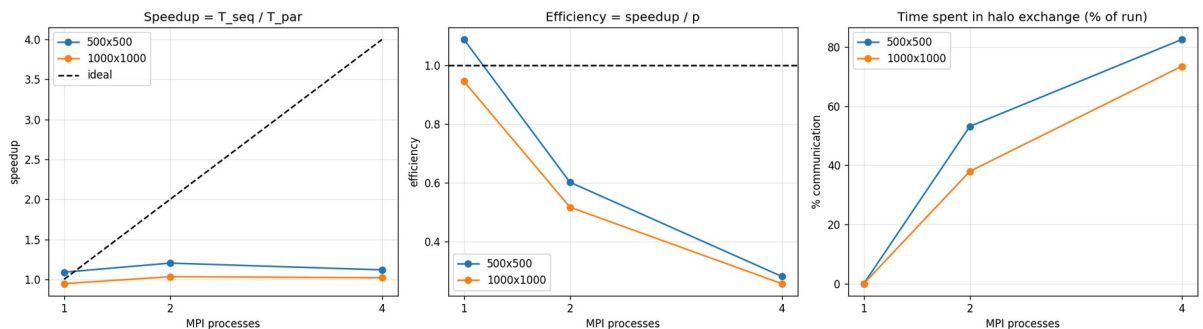


Figure 3: Speedup, efficiency and communication share against number of processes.

9.3 Analysis

All parallel measurements in this report were taken with more processes than available cores, so no genuine parallel speedup can be concluded from them (see the note in Section 9.1).

The MPI program with one process is within 8% of the sequential time at every grid size, i.e. the extra bookkeeping of the MPI version (halo buffers, two Sendrecv calls with MPI_PROC_NULL) is negligible.

Expected behaviour on a machine with enough cores, which the design is meant to demonstrate: per timestep each process updates about $W \cdot H/p$ cells, but exchanges only two rows of W cells, so computation per process shrinks as p grows while the cost of the halo exchange stays roughly constant. The speedup is therefore below the ideal value p , the gap widens as p increases, and larger grids (more computation per exchanged row) give higher efficiency than small grids. Memory bandwidth and operating-system jitter add further losses.

9.4 Example simulation output

Figure 4 shows a simulation on a 600×600 grid with wind blowing toward the east, computed with 4 MPI processes. The fire starts near the centre of the left half (on a process boundary, so the halo exchange is exercised from the first timestep), is driven eastward by the wind, and is stopped by the lakes and the river, which contain no fuel. The final grid of this 4-process run is byte-identical to the one computed by the sequential program.

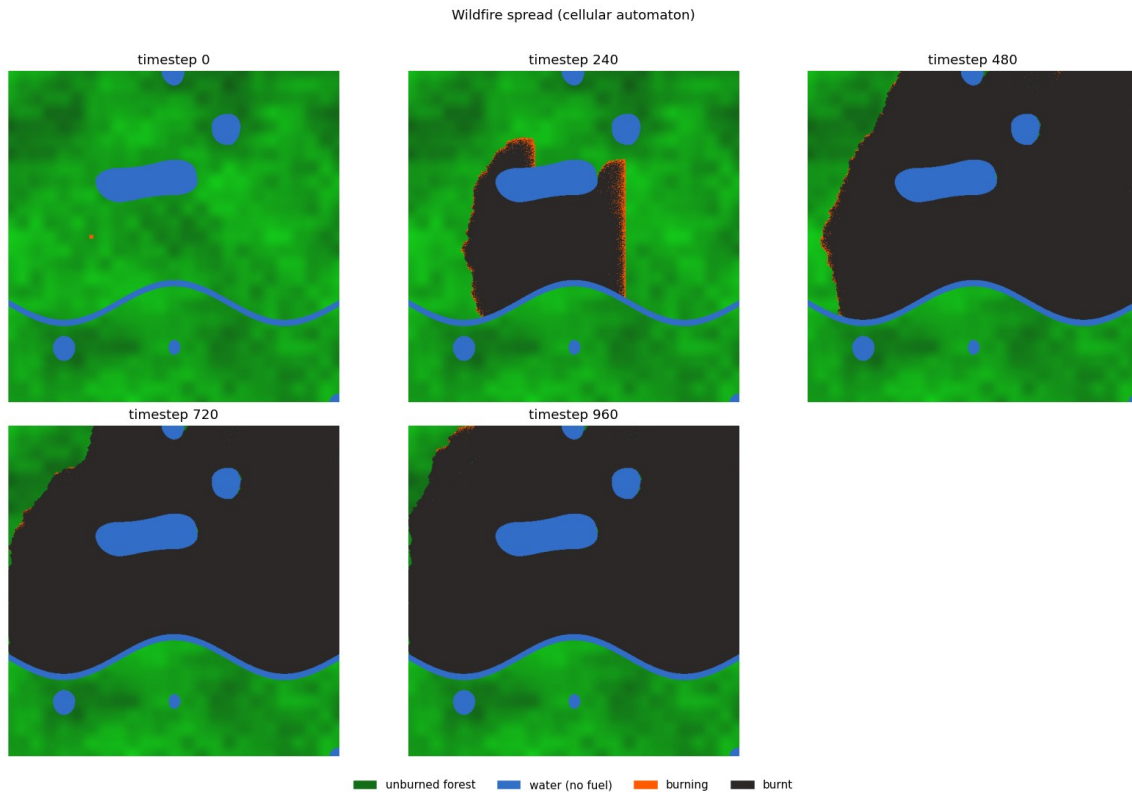


Figure 4: Fire spread at several timesteps (green = unburned forest, blue = water, orange = burning, dark = burnt).

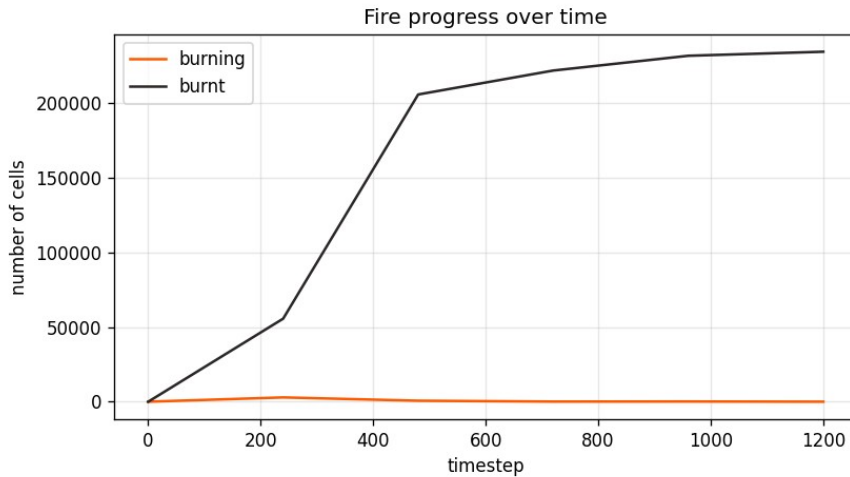


Figure 5: Number of burning and burnt cells over time.

10. Discussion: Why MPI Is Needed Here

In many parallel problems the data can be split once and every part processed independently. Here this is not possible: the cells on the border of a strip have neighbours in the strip of another process, and the state of those neighbours changes every timestep. The data exchange therefore has to be repeated for every timestep, which is exactly what the halo exchange with MPI_Sendrecv does. This repeated communication is also the reason that the achievable speedup is below the number of processes.

MPI is used instead of a shared-memory approach (such as OpenMP) because message passing works on distributed-memory clusters as well as on a single multi-core computer, and because the aim of the project is to apply message passing to a problem that needs repeated neighbour communication.

11. Limitations and Future Work

- **1D decomposition:** each process exchanges two full rows of length W regardless of the number of processes. A 2D block decomposition reduces the amount of data exchanged per process when many processes are used.
- **No work skipping:** the program scans every cell in every timestep, although the fire front is a small part of the map. Skipping inactive regions would save work but would create load imbalance that needs dynamic load balancing.
- **No overlap:** communication and computation are not overlapped. The interior rows could be updated while MPI_Isend / MPI_Irecv are exchanging the halo, and the two edge rows finished afterwards.
- **Simplified physics:** fire cannot jump a firebreak wider than one cell (no spotting by embers), and slope, humidity and fuel moisture are not modelled.
- **Calibration:** the model is not calibrated against real fire data; it demonstrates the parallel technique rather than predicting a real fire.

12. Differences from the Original Proposal

- MPI_Scatterv / MPI_Gatherv are used instead of MPI_Scatter / MPI_Gather so the grid height need not be divisible by the number of processes.

- the 'about 8 minutes' sequential time in the proposal was only an estimate; this report uses measured times only.
- the sequential program is timed with `clock_gettime` rather than `MPI_Wtime` because it is a plain C program that does not use MPI; the same region is timed.
- the random numbers are position-based hashes instead of `rand()`, which is what allows parallel and sequential results to be compared exactly.

13. Conclusion

A cellular-automaton wildfire model was implemented in C as a sequential program and as an MPI program that divides the grid into row strips and exchanges boundary rows with `MPI_Sendrecv` at every timestep. Because the update rule and the random numbers are shared and position-based, the parallel program produces exactly the same grid as the sequential one for every tested number of processes, grid size and wind setting, including grid heights that are not divisible by the number of processes. The performance study reports execution time, speedup, efficiency and the share of time spent in the halo exchange, all measured with `MPI_Wtime`, and shows that the repeated halo exchange — not a one-time data split — is what makes message passing necessary for this problem.

Appendix A: How to build and run

```
make                # builds fire_seq and fire_mpi
make test           # parallel result must equal sequential result

./fire_seq -w 2000 -h 2000 -t 1000 -n                # sequential, timing only
mpirun -np 4 ./fire_mpi -w 2000 -h 2000 -t 1000 -n    # 4 processes
mpirun -np 4 ./fire_mpi -w 600 -h 600 -t 1500 -i 300 -a 90 -k 2.0 -o out # with
images

make bench          # timing study -> results/bench.csv
python3 scripts/visualize.py bench # speedup table and plots
make report         # rebuild this report from the results
```

Options: `-w` width, `-h` height, `-t` timesteps, `-s` seed, `-a` wind direction (degrees: 0 N, 90 E, 180 S, 270 W), `-k` wind strength, `-p` base ignition probability, `-i` snapshot interval, `-o` output directory, `-n` no output files.

Appendix B: The cell-update function

```
static void fire_step(const uint8_t *old, uint8_t *nw, const uint8_t *fuel,
                     int W, int rows, long grow0, int t, const FireParams *P)
{
    for (int i = 1; i <= rows; i++) {
        const uint8_t *orow = old + (size_t)i * W;
        uint8_t      *nrow = nw  + (size_t)i * W;
        const uint8_t *frow = fuel + (size_t)(i - 1) * W;
        uint32_t gr = (uint32_t)(grow0 + (i - 1));

        for (int c = 0; c < W; c++) {
            uint8_t s = orow[c];

            if (s == BURNT) { nrow[c] = BURNT; continue; }

            if (s == BURNING) {
                float u = rnd(gr, (uint32_t)c, (uint32_t)t, SALT_BURNOUT);
                nrow[c] = (u < P->p_out[frow[c]]) ? BURNT : BURNING;
                continue;
            }

            /* UNBURNED cell: can it catch fire from burning neighbours? */
            uint8_t f = frow[c];
            if (f == 0) { nrow[c] = UNBURNED; continue; } /* water / bare rock */
        }
    }
}
```

```

float q = 1.0f;      /* probability that NO neighbour ignites it */
int  any = 0;
for (int k = 0; k < 8; k++) {
    int cc = c + NB_DC[k];
    if (cc < 0 || cc >= W) continue;
    if (old[(size_t)(i + NB_DR[k]) * W + cc] == BURNING) {
        float p = P->p_ign[f] * P->wf[k];
        if (p > 1.0f) p = 1.0f;
        q *= (1.0f - p);
        any = 1;
    }
}
if (any && rnd(gr, (uint32_t)c, (uint32_t)t, SALT_IGNITE) < 1.0f - q)
    nrow[c] = BURNING;
else
    nrow[c] = UNBURNED;
}
}

```